Name:S.Suhas Assignment-16.2

Ht.no:2303A51453

Course:AIAC Bt.no:21

Task Descrption-1: (Design Database Schema for Student Management System)

Instructions:

• Using AI tools, generate CREATE TABLE statements for a Student Management Database.

• Tables: Students, Courses, Enrollments.

• Include primary keys, foreign keys, and suitable data types.

Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Sample Code:

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100),
```

credits INT

);

CREATE TABLE Enrollments (

enrollment_id INT PRIMARY KEY,

student_id INT,

course_id INT,

FOREIGN KEY (student_id) REFERENCES Students(student_id),

FOREIGN KEY (course_id) REFERENCES Courses(course_id)

);

Expected Output:

Tables Students, Courses, and Enrollments are created successfully.

Foreign key constraints are properly enforced.

Task Description- 2: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Sample Code:

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);
```

Expected Output:

- Sample records are inserted successfully.
- FROM Students; displays all student records.
- SELECT * FROM Enrollments; displays all enrollments.

```sql
-- Create Students table
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    -- Additional fields to demonstrate 3NF compliance
    -- (separating student personal details from enrollment data)
    phone VARCHAR(15),
    address VARCHAR(200)
);

-- Create Courses table
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT NOT NULL CHECK (credits > 0),
    -- Department field added to show 2NF compliance
    -- (credits depend on course_id, not on any partial key)
    department VARCHAR(50)
);

-- Create Enrollments table (junction table for many-to-many relationship)
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrollment_date DATE DEFAULT CURRENT_DATE,
    grade CHAR(2),
    -- Foreign key constraints
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCAD
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE,
    -- Ensure no duplicate enrollments
    UNIQUE (student_id, course_id)
);
```

Expected-Output:

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
-- Additional fields to demonstrate 3NF compliance
-- (separating student personal details from enrollment data)
phone VARCHAR(15),
address VARCHAR(200)
);
```

```
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100) NOT NULL,
credits INT NOT NULL CHECK (credits > 0),
-- Department field added to show 2NF compliance
-- (credits depend on course_id, not on any partial key)
department VARCHAR(50)
);
```

```
CREATE TABLE Enrollments (
enrollment_id INT PRIMARY KEY,
student_id INT NOT NULL,
course_id INT NOT NULL,
enrollment_date DATE DEFAULT CURRENT_DATE,
grade CHAR(2),
-- Foreign key constraints
FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE,
FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE,
-- Ensure no duplicate enrollments
UNIQUE (student_id, course_id)
);
```

Task Description- 2: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Sample Code:

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);
```

Expected Output:

• Sample records are inserted successfully.

• FROM Students; displays all student records.

- SELECT * FROM Enrollments; displays all enrollments.

```sql
-- Insert sample students
INSERT INTO Students (student_id, name, email, phone, address) VALUES
(1, 'Arjun', 'arjun@gmail.com', '9876543210', 'Mumbai, India'),
(2, 'Meera', 'meera@gmail.com', '9876543211', 'Delhi, India'),
(3, 'Rahul', 'rahul@gmail.com', '9876543212', 'Bangalore, India');

-- Insert sample courses
INSERT INTO Courses (course_id, course_name, credits, department) VALUES
(101, 'Database Systems', 4, 'Computer Science'),
(102, 'Data Structures', 3, 'Computer Science'),
(103, 'Operating Systems', 4, 'Information Technology'),
(104, 'AI Fundamentals', 3, 'Artificial Intelligence');

-- Insert sample enrollments
INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_dat
(1, 1, 101, '2024-01-15', 'A'),
(2, 1, 102, '2024-01-15', 'B+'),
(3, 2, 103, '2024-01-16', 'A-'),
(4, 3, 101, '2024-01-16', 'B'),
(5, 3, 104, '2024-01-17', 'A');

-- Verify the data insertion with SELECT queries
```

```
- Display all students
ELECT * FROM Students;
* Expected Output:
tudent_id | name  | email           | phone     | address
          | Arjun | arjun@gmail.com | 9876543210| Mumbai, India
          | Meera | meera@gmail.com | 9876543211| Delhi, India
          | Rahul | rahul@gmail.com | 9876543212| Bangalore, India
/

- Display all courses
ELECT * FROM Courses;
* Expected Output:
ourse_id | course_name         | credits | department
01       | Database Systems    | 4       | Computer Science
02       | Data Structures     | 3       | Computer Science
03       | Operating Systems   | 4       | Information Technology
04       | AI Fundamentals     | 3       | Artificial Intelligence
/

- Display all enrollments
ELECT * FROM Enrollments;
* Expected Output:
nrollment_id | student_id | course_id | enrollment_date | grade
             | 1          | 101       | 2024-01-15      | A
             | 1          | 102       | 2024-01-15      | B+
             | 2          | 103       | 2024-01-16      | A-
             | 3          | 101       | 2024-01-16      | B
             | 3          | 104       | 2024-01-17      | A
/
```

Expected-Output:

```
INSERT INTO Students (student_id, name, email, phone, address) VALUES
(1, 'Arjun', 'arjun@gmail.com', '9876543210', 'Mumbai, India'),
(2, 'Meera', 'meera@gmail.com', '9876543211', 'Delhi, India'),
(3, 'Rahul', 'rahul@gmail.com', '9876543212', 'Bangalore, India');
```

```
INSERT INTO Courses (course_id, course_name, credits, department) VALUES
(101, 'Database Systems', 4, 'Computer Science'),
(102, 'Data Structures', 3, 'Computer Science'),
(103, 'Operating Systems', 4, 'Information Technology'),
(104, 'AI Fundamentals', 3, 'Artificial Intelligence');
```

```
INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date, grade)
VALUES
(1, 1, 101, '2024-01-15', 'A'),
(2, 1, 102, '2024-01-15', 'B+'),
(3, 2, 103, '2024-01-16', 'A-'),
(4, 3, 101, '2024-01-16', 'B'),
(5, 3, 104, '2024-01-17', 'A');
```

```
SELECT * FROM Students;
```

student_id	name	email	phone	address
1	Arjun	arjun@gmail.com	9876543210	Mumbai, India
2	Meera	meera@gmail.com	9876543211	Delhi, India
3	Rahul	rahul@gmail.com	9876543212	Bangalore, India

```
SELECT * FROM Courses;
```

course_id	course_name	credits	department
101	Database Systems	4	Computer Science
102	Data Structures	3	Computer Science
103	Operating Systems	4	Information Technology
104	AI Fundamentals	3	Artificial Intelligence

```
SELECT * FROM Enrollments;
```

enrollment_id	student_id	course_id	enrollment_date	grade
1	1	101	2024-01-15	A
2	1	102	2024-01-15	B+
3	2	103	2024-01-16	A-
4	3	101	2024-01-16	B
5	3	104	2024-01-17	A

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- *Retrieve all courses*
- *Update a student's email*

• *Delete an enrollment record*

• *Validate the changes using SELECT statements.*

Sample Code:

```
SELECT * FROM Courses;

UPDATE Students

SET email = 'arjun_new@gmail.com'

WHERE student_id = 1;

DELETE FROM Enrollments

WHERE enrollment_id = 5;
```

Expected Output:

• *Updated email is reflected in Students table.*

• *Deleted enrollment record is removed successfully.*

• *Remaining data remains unchanged.*

```sql
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL
);

CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL
);
```

```sql
-- Insert initial data
INSERT INTO Students VALUES
(1, 'Arjun', 'Sharma', 'arjun@email.com'),
(2, 'Priya', 'Patel', 'priya@email.com');

INSERT INTO Courses VALUES
(101, 'Mathematics'),
(102, 'Computer Science');

INSERT INTO Enrollments VALUES
(5, 1, 101),
(6, 2, 102);
```

```sql
-- ============================================
-- PERFORM CRUD OPERATIONS
-- ============================================

-- Retrieve all courses
SELECT * FROM Courses;

-- Update student email
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;

-- Delete enrollment record
DELETE FROM Enrollments
WHERE enrollment_id = 5;
```

```sql
-- ============================================
-- EXPECTED OUTPUT - VALIDATE CHANGES
-- ============================================

-- 1. Verify updated email is reflected in Students table
SELECT '--- STUDENTS TABLE (After Update) ---' AS ' ';
SELECT * FROM Students;
/* EXPECTED OUTPUT:
student_id | first_name | last_name | email
1          | Arjun      | Sharma    | arjun_new@gmail.com  (email updated)
2          | Priya      | Patel     | priya@email.com      (unchanged)
*/

-- 2. Verify deleted enrollment record is removed
SELECT '--- ENROLLMENTS TABLE (After Delete) ---' AS ' ';
SELECT * FROM Enrollments;
/* EXPECTED OUTPUT:
enrollment_id | student_id | course_id
6             | 2          | 102       (record 5 is deleted)
*/
```

```sql
-- 3. Verify remaining data remains unchanged
SELECT '--- COURSES TABLE (Unchanged) ---' AS ' ';
SELECT * FROM Courses;
/* EXPECTED OUTPUT:
course_id | course_name
101       | Mathematics    (unchanged)
102       | Computer Science (unchanged)
*/
```

Expected-Output:

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL
);
```

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY,
course_name TEXT NOT NULL
);
```

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL
);
```

```sql
INSERT INTO Students VALUES
(1, 'Arjun', 'Sharma', 'arjun@email.com'),
(2, 'Priya', 'Patel', 'priya@email.com');
```

```sql
INSERT INTO Courses VALUES
(101, 'Mathematics'),
(102, 'Computer Science');
```

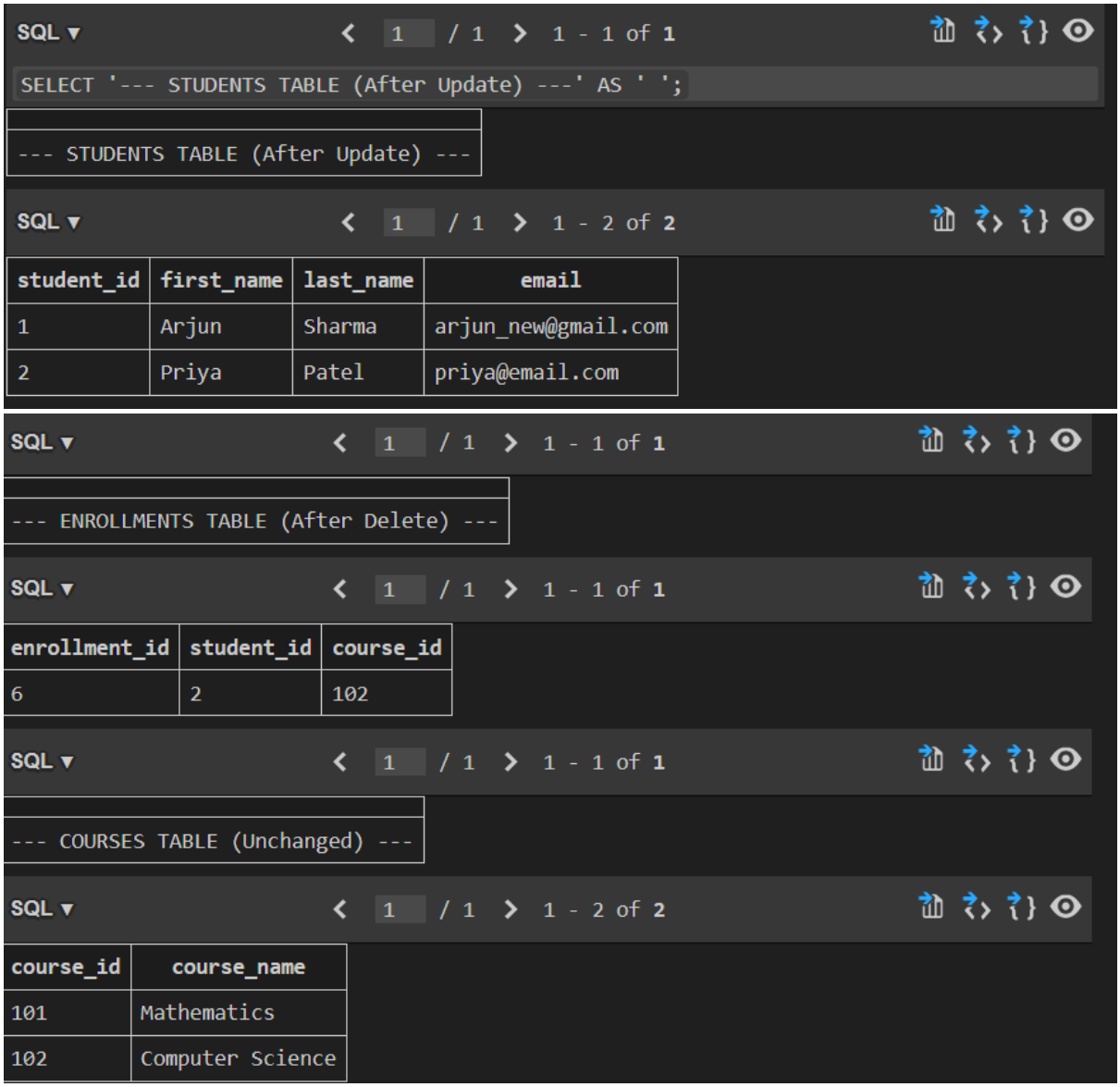
```
INSERT INTO Enrollments VALUES
(5, 1, 101),
(6, 2, 102);
```

```
SELECT * FROM Courses;
```

course_id	course_name
101	Mathematics
102	Computer Science

```
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
```

```
DELETE FROM Enrollments
WHERE enrollment_id = 5;
```



Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

• Count number of students enrolled per course

• Display courses with more than one student enrolled

Analyze the correctness of results.

Sample Code:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Expected Output:

- Correct count of students per course is displayed.
- Courses with multiple enrollments are listed accurately.

```sql
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY,
    course_name TEXT NOT NULL
);

CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL
);
```

```sql
-- Insert sample data with multiple enrollments
INSERT INTO Students VALUES
(1, 'Arjun', 'Sharma', 'arjun@email.com'),
(2, 'Priya', 'Patel', 'priya@email.com'),
(3, 'Rahul', 'Verma', 'rahul@email.com'),
(4, 'Neha', 'Singh', 'neha@email.com');

INSERT INTO Courses VALUES
(101, 'Mathematics'),
(102, 'Computer Science'),
(103, 'Physics');

INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 2, 101),
(3, 3, 101),
(4, 1, 102),
(5, 2, 102),
(6, 4, 103);
```

```sql
-- ============================================
-- AGGREGATE QUERY 1: Count students per course
-- ============================================

SELECT '--- QUERY 1: COUNT STUDENTS PER COURSE ---' AS ' ';

SELECT
    c.course_name,
    COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
ORDER BY student_count DESC;

/* EXPECTED OUTPUT:
course_name         | student_count
Mathematics         | 3
Computer Science    | 2
Physics             | 1
*/
```

```sql
-- ============================================
-- AGGREGATE QUERY 2: Courses with more than one student
-- ============================================

SELECT '--- QUERY 2: COURSES WITH >1 STUDENT ---' AS ' ';

SELECT
    c.course_name,
    COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;

/* EXPECTED OUTPUT:
course_name         | student_count
Mathematics         | 3
Computer Science    | 2
*/
```

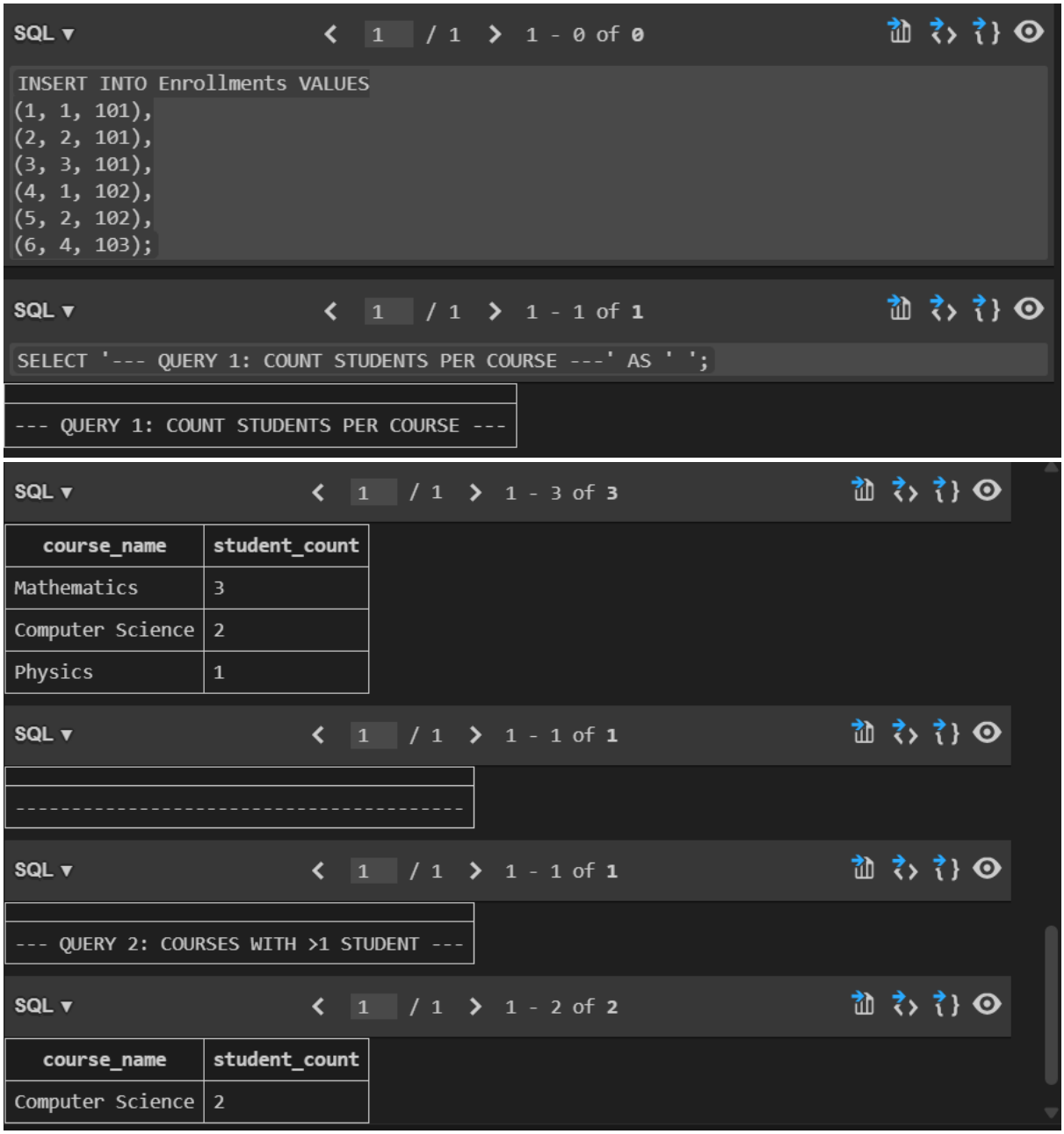
```
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL
);
```

```
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY,
course_name TEXT NOT NULL
);
```

```
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL
);
```

```
INSERT INTO Students VALUES
(1, 'Arjun', 'Sharma', 'arjun@email.com'),
(2, 'Priya', 'Patel', 'priya@email.com'),
(3, 'Rahul', 'Verma', 'rahul@email.com'),
(4, 'Neha', 'Singh', 'neha@email.com');
```

```
INSERT INTO Courses VALUES
(101, 'Mathematics'),
(102, 'Computer Science'),
(103, 'Physics');
```



Task Description- 5: (Query Optimization)

Instructions:

• Use AI tools to analyze query performance.

• Rewrite inefficient sub queries using JOINs and test both queries

for correctness.

Sample Code:

```
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Expected Output:

- Both queries return the same set of students.
- Optimized query performs better on large datasets.

This lab allows students to use AI tools like Copilot, Cursor, or Gemini to:

- Generate schema definitions
- Auto-complete insert statements
- Write CRUD and aggregate SQL queries
- Optimize queries and ensure proper normalization

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL
);
```

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY,
course_name TEXT NOT NULL
);
```

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

```sql
INSERT INTO Students VALUES
(1, 'Arjun', 'Sharma', 'arjun@email.com'),
(2, 'Priya', 'Patel', 'priya@email.com'),
(3, 'Rahul', 'Verma', 'rahul@email.com'),
(4, 'Neha', 'Singh', 'neha@email.com');
```

```sql
INSERT INTO Courses VALUES
(101, 'Mathematics'),
(102, 'Computer Science'),
(103, 'Physics');
```

```
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 2, 101),
(3, 3, 101),
(4, 1, 102),
(5, 2, 102),
(6, 4, 103);
```

SQL ▾ < 1 / 1 > 1 - 0 of 0

```
CREATE INDEX idx_enrollments_course ON Enrollments(course_id);
```

SQL ▾ < 1 / 1 > 1 - 0 of 0

```
CREATE INDEX idx_enrollments_student ON Enrollments(student_id);
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
SELECT '--- ORIGINAL QUERY (Subquery) ---' AS ' ';
```

```
--- ORIGINAL QUERY (Subquery) ---
```

```
Finding students enrolled in course 101:
```

SQL ▾ < 1 / 1 > 1 - 3 of 3

student_id	first_name	last_name	email
1	Arjun	Sharma	arjun@email.com
2	Priya	Patel	priya@email.com
3	Rahul	Verma	rahul@email.com

```
--- OPTIMIZED QUERY (JOIN) ---
```

SQL ▾ < 1 / 1 > 1 - 1 of 1

```
Same students using JOIN:
```

SQL ▾ < 1 / 1 > 1 - 3 of 3

student_id	first_name	last_name	email
1	Arjun	Sharma	arjun@email.com
2	Priya	Patel	priya@email.com
3	Rahul	Verma	rahul@email.com